

DEX-AR: the equations, before and after the rank prefilter

This note states what DEX-AR computes as published, then states the one line that changes. Everything here is transcribed from the implementation in `dexar/wrapper.py`; equation numbers in parentheses refer to the paper (Bousselham et al., *DEX-AR: A Dynamic Explainability Method for Autoregressive VLMs*, arXiv:2603.06302).

1 Notation

For a generation step t , layer $l \in \{1, \dots, L\}$ and attention head $i \in \{1, \dots, H\}$:

$\mathcal{V}, \mathcal{T}$	the image-token and text-token positions of the sequence; $N = \mathcal{V} = h \times w$
$A^{l,t}$	attention probabilities at layer l , step t
y_t^l	hidden state at the last position, layer l
$o^{l,t}$	intermediate logits, $o^{l,t} = \text{LM_Head}(y_t^l)$ (Eq. 3, the logit lens)
v_t	the sampled token at step t : the final layer's argmax (Eq. 2)

DEX-AR differentiates the sampled token's intermediate logit with respect to the attention probabilities, and keeps only the last query row — the row belonging to the position that is about to emit v_t :

$$G^{l,t,i}[j] = \left(\frac{\partial o_{v_t}^{l,t}}{\partial A_i^{l,t}[-1, j]} \right)^+, \quad j \in \mathcal{V}, \quad (1)$$

and the same quantity over $j \in \mathcal{T}$, written $G_{\text{txt}}^{l,t,i}$. Each $G^{l,t,i}$ is a map over the N image positions, reshaped to the token grid $h \times w$. Because $\partial o / \partial A_{[:,j]} = \langle v_j, g \rangle$, this map carries a factor of the value-vector norm $\|v_j\|$ but no factor of A_j itself: DEX-AR reads counterfactual sensitivity, not realised usage.

2 DEX-AR as published

Head scores (Eq. 4). Two scalars per head, each a maximum over the columns of the respective block. These produce no heatmap; they are a scoring step.

$$S_{\text{img}}^{l,t,i} = \max_{j \in \mathcal{V}} G^{l,t,i}[j], \quad (2)$$

$$S_{\text{txt}}^{l,t,i} = \max_{j \in \mathcal{T}} G_{\text{txt}}^{l,t,i}[j]. \quad (3)$$

Head filter and per-token map (Eq. 5). Each head is weighted by how much more it attends to image than to text, and the weighted maps are summed over *all* L layers and all H heads:

$$w^{l,t,i} = \left(S_{\text{img}}^{l,t,i} - S_{\text{txt}}^{l,t,i} \right)^+, \quad (4)$$

$$E^{(t)} = \sum_{l=1}^L \sum_{i=1}^H w^{l,t,i} \cdot G^{l,t,i}. \quad (5)$$

Token filter and sentence map (Eq. 6). The per-token maps are aggregated under a token weight that applies the same difference to the *global* maxima, taken across every layer and head:

$$\delta^t = \left(\max_{l,i} S_{\text{img}}^{l,t,i} - \max_{l,i} S_{\text{txt}}^{l,t,i} \right)^+, \quad (6)$$

$$\bar{E} = \sum_{t=1}^T \delta^t \cdot E^{(t)}. \quad (7)$$

Where the weight comes from. Both filters are decided by $\max_p G(p)$ — a head’s single tallest cell, and nothing else about the shape of its map. Two heads with the same peak score identically whether that peak sits in a coherent region or alone.

3 After: the rank prefilter

The operator. Let R_k take, at every cell, the k -th order statistic of that cell’s 3×3 neighbourhood on the token grid, with replicate padding at the border:

$$(R_k G)(p) = \text{sort} \left(\left\{ G(q) : q \in \mathcal{N}_{3 \times 3}(p) \right\} \right)_k, \quad k \in \{0, \dots, 8\}. \quad (8)$$

$k = 0$ is erosion, $k = 4$ the median, $k = 8$ dilation. Border cells replicate rather than zero-pad: zero padding would erode the border as a padding artifact, which on a model with a border-peak bias would look like the method working.

Why it targets (2). An isolated spike cannot survive a sub-median R_k — its neighbourhood holds eight small values — while a coherent region can. Applying R_k *before* (2) rewrites the head score from “the tallest cell” into “the tallest cell that has neighbours”, without touching the form of Eq. 5 or Eq. 6.

The rescale. S_{txt} lives on the one-dimensional token sequence and receives no matching operation. Left alone, the subtraction in (4) would compare a filtered quantity against an unfiltered one and the head weights would collapse. So S_{img} is rescaled per (t, l) by the ratio of medians over heads:

$$c^{l,t} = \frac{\text{median}_i \max_p G^{l,t,i}(p)}{\text{median}_i \max_p (R_k G^{l,t,i})(p)}. \quad (9)$$

The two lines that change. Equations (1), (3), (4), (5), (6) and (7) are untouched. Only these are new:

$$\tilde{G}^{l,t,i} = R_k G^{l,t,i}, \quad (1')$$

$$\tilde{S}_{\text{img}}^{l,t,i} = c^{l,t} \cdot \max_{j \in \mathcal{V}} \tilde{G}^{l,t,i}[j], \quad (2')$$

with $\tilde{G}$ and $\tilde{S}_{\text{img}}$ substituted for G and S_{img} everywhere downstream. Setting k to nothing recovers the published method exactly: the prefilter is off by default.

4 Metrics

All reported $\times 100$. A is the map bilinearly upsampled to mask resolution and min-max normalised; M is the ground-truth mask (the union of every object present).

$\text{soft-IoU} = \frac{\sum AM}{\sum A + \sum M - \sum AM}$	threshold-free
$\text{IoU} = \max_{\tau \in \{\tau_1 \dots \tau_{20}\}} \text{IoU}(A > \tau, M)$	best of $k = 20$ thresholds, per prediction
$\text{EPG} = \frac{\sum AM}{\sum A}$	share of attribution energy inside the mask
$\text{SNR} = 10 \log_{10} \left(\frac{\langle A, M \rangle / \ M\ _1}{\langle A, 1 - M \rangle / \ 1 - M\ _1} \right)$	0 for a uniform map

Soft-IoU and EPG both rise with object area, so neither separates signal from noise on its own: on the 1449-image PascalVOC val split a random smooth map scores soft-IoU 18.41 / IoU 25.46 / EPG 25.24 / SNR -0.01 , and a uniform map scores 25.24 on all three of the paper’s metrics. SNR is the only one of the four with a defined zero. Recompute the chance lines for any other split with `experiments/voc_nulls.py`.

5 Where this lives in the code

(1), (2), (3)	dexar/wrapper.py, the per-layer loop in <code>compute_dexar</code>
(8)	dexar/wrapper.py, <code>DexarWrapper._rank_filter</code>
(9)	dexar/wrapper.py, the <code>map_filter</code> branch
(4), (5)	dexar/wrapper.py, “Head filtering (Eq. 5)”
(6), (7)	dexar/wrapper.py, sentence aggregation
metrics	experiments/eval_masks.py, <code>score()</code>

The prefilter is reached as `compute_dexar(..., map_filter="rank:k")`, or `MAP_FILTER=rank:k` for `eval_masks.py`. `rank:k:noscale` ablates (9). `drop_delta` sets $\delta^t = 1$, which switches off (6).